

Linux■■■■■■■■■■

■■■■■

■■■■20231035109

■■■■■■■■■


```

echo "$username:$password" | sudo chpasswd
echo -e "${GREEN}[██] ██ '$username' █████${NC}"
write_log "██████: $username"
# █████
echo -e "\n${YELLOW}--- █████ ---${NC}"
id "$username"
echo "████: /home/$username"
else
echo -e "${RED}[██] ████████${NC}"
write_log "██████: $username"
return 1
fi
}
# =====
# ███2███/██████
# █████
# 1. ██████████
# 2. ██████████
# 3. ███chpasswd██████
# 4. ████████████████
# =====
change_password() {
echo -e "${BLUE}=====${NC}"
echo -e "${BLUE} ████████${NC}"
echo -e "${BLUE}=====${NC}"
read -p "██████████: " username
# ████████
if ! id "$username" &>/dev/null; then
echo -e "${RED}[██] ██ '$username' █████${NC}"
return 1
fi
# ████████
read -s -p "██████: " new_password
echo ""
read -s -p "██████: " confirm_password
echo ""
# ████████
if [ "$new_password" != "$confirm_password" ]; then
echo -e "${RED}[██] ██████████${NC}"
return 1
fi
# ████████
echo -e "${YELLOW}[██████]...${NC}"
echo "$username:$new_password" | sudo chpasswd
if [ $? -eq 0 ]; then
echo -e "${GREEN}[██] ██ '$username' ████████${NC}"
write_log "██████ $username █████"
else
echo -e "${RED}[██] ████████${NC}"
return 1
fi
}
# =====
# ███3███████████████
# ████████
# 1. ██████████
# 2. ███etc/shadow███████████████
# 3. ████████3███████
# 4. ██████████
# =====
user_login() {
echo -e "${BLUE}=====${NC}"
echo -e "${BLUE} ████████${NC}"
echo -e "${BLUE}=====${NC}"
read -p "██████: " username
# ████████
if ! id "$username" &>/dev/null; then
echo -e "${RED}[██] ██ '$username' █████${NC}"
return 1
fi
# ████████████████3███
local max_attempts=3
local attempt=1
while [ $attempt -le $max_attempts ]; do
echo -e "${YELLOW}█ $attempt/$max_attempts █████${NC}"
read -s -p "██████: " input_password

```

```

echo ""
# #####
# #####
# ##su#####root#####sudo##
echo "$input_password" | sudo -S su - "$username" -c "echo 'success'" 2>/dev/null
if [ $? -eq 0 ]; then
echo -e "${GREEN}===== ${NC}"
echo -e "${GREEN} #####$username${NC}"
echo -e "${GREEN}===== ${NC}"
write_log "## $username ##### ($attempt###)"
# #####
echo -e "\n${YELLOW}--- ##### ---${NC}"
echo "####: $username"
echo "UID: $(id -u $username)"
echo "##ID: $(id -g $username)"
echo "####: $(grep "^$username:" /etc/passwd | cut -d: -f6)"
echo "##Shell: $(grep "^$username:" /etc/passwd | cut -d: -f7)"
echo "#####: $(date '+%Y-%m-%d %H:%M:%S')"
return 0
else
attempt=$((attempt + 1))
remaining=$((max_attempts - attempt + 1))
if [ $attempt -le $max_attempts ]; then
echo -e "${RED}[##] ##### $remaining ####${NC}"
fi
fi
done
# #####
echo -e "${RED}===== ${NC}"
echo -e "${RED} #####(3)${NC}"
echo -e "${RED}===== ${NC}"
write_log "WARNING: ## $username #####3#####"
return 1
}
# =====
# ##4#####
# #####
# 1. #####
# 2. #####
# 3. #####
# 4. #####
# =====
delete_user() {
echo -e "${BLUE}===== ${NC}"
echo -e "${_blue} ####${NC}"
echo -e "${BLUE}===== ${NC}"
read -p "#####: " username
# #####
if ! id "$username" &>/dev/null; then
echo -e "${RED}[##] ## '$username' #####${NC}"
return 1
fi
# #####root##
current_user=$(whoami)
if [ "$username" = "root" ] || [ "$username" = "$current_user" ]; then
echo -e "${RED}[##] #####${NC}"
return 1
fi
# #####
echo -e "${YELLOW}[##] #####${NC}"
read -p "##### '$username' ##(yes/no): " confirm
if [ "$confirm" != "yes" ]; then
echo -e "${YELLOW}#####${NC}"
return 0
fi
# #####
read -p "#####(y/n): " del_home
echo -e "${YELLOW}[#####]...${NC}"
if [ "$del_home" = "y" ] || [ "$del_home" = "Y" ]; then
# -r#####
sudo userdel -r "$username"
home_msg="####"
else
# #####
sudo userdel "$username"
home_msg="(#####)"

```

```

fi
if [ $? -eq 0 ]; then
echo -e "${GREEN}[██] ██ '$username' █████ $home_msg${NC}"
write_log "██████: $username $home_msg"
else
echo -e "${RED}[██] ████████${NC}"
write_log "██████: $username"
return 1
fi
}
# =====
# ████████████████████
# ████████
# 1. ████/etc/passwd██████UID>=1000██████
# 2. ████████████████
# =====
list_users() {
echo -e "${BLUE}===== ${NC}"
echo -e "${BLUE} ████████ ${NC}"
echo -e "${BLUE}===== ${NC}"
echo ""
# ████████
printf "%-15s %-8s %-20s %-15s\n" "████" "UID" "████" "Shell"
printf "%-15s %-8s %-20s %-15s\n" "-----" "-----" "-----" "-----"
# ████awk██████UID >= 1000██
# /etc/passwd██: █████:UID:GID:██:████:Shell
awk -F: '$3 >= 1000 {printf "%-15s %-8s %-20s %-15s\n", $1, $3, $6, $7}' /etc/passwd
echo ""
echo -e "${YELLOW}█ $(awk -F: '$3 >= 1000' /etc/passwd | wc -l) ██████ ${NC}"
}
# =====
# ████████
# ████████
# 1. ████████████████████
# 2. ████case███████████████
# 3. ████████████████
# =====
main_menu() {
init_log
while true; do
echo ""
echo -e "${GREEN}██████████████████████████████████████████████████████████████ ${NC}"
echo -e "${GREEN}█ Linux ████████ v1.0 █ ${NC}"
echo -e "${GREEN}██████████████████████████████████████████████████████████████████████████ ${NC}"
echo -e "${GREEN}█ █ ${NC}"
echo -e "${GREEN}█ 1. ██████ █ ${NC}"
echo -e "${GREEN}█ 2. ████████ █ ${NC}"
echo -e "${GREEN}█ 3. ████████ █ ${NC}"
echo -e "${GREEN}█ 4. ██████ █ ${NC}"
echo -e "${GREEN}█ 5. ████████ █ ${NC}"
echo -e "${GREEN}█ 0. ██████ █ ${NC}"
echo -e "${GREEN}█ █ ${NC}"
echo -e "${GREEN}██████████████████████████████████████████████████████████████████████████ ${NC}"
echo ""
read -p "██████ (0-5): " choice
# case██████████
case $choice in
1)
create_user
;;
2)
change_password
;;
3)
user_login
;;
4)
delete_user
;;
5)
list_users
;;
0)
echo -e "${YELLOW}██████████████████████████████████████████████████████████████ ${NC}"
write_log "██████████"
exit 0

```

```
;;
*)
echo -e "${RED}[██] ████████████████████${NC}"
;;
esac
# ████████████████████
echo ""
read -p "██████████..."
done
}
# ████████████████████
main_menu
```



```

# 1.
if [ $? -eq 0 ]; then
# 1.755
chmod 755 "$folder_path"
echo -e "${GREEN}[1] 1.755 '$folder_path' 1.755$parent_info${NC}"
# 1.755
echo -e "\n${YELLOW}--- 1.755 ---${NC}"
ls -ld "$folder_path"
log_operation "1.755" "$folder_path" "1.755"
else
echo -e "${RED}[1] 1.755$parent_info${NC}"
log_operation "1.755" "$folder_path" "1.755-1.755"
return 1
fi
}
# =====
# 2.
# 2.
# 1. 2.
# 2. 2.
# 3. 2.
# 4. 2.
# =====
delete_folder() {
echo -e "${BLUE}===== ${NC}"
echo -e "${BLUE} 2. ${NC}"
echo -e "${BLUE}===== ${NC}"
read -p "2. : " folder_path
# 2.
if [ ! -d "$folder_path" ]; then
echo -e "${RED}[2] 2. '$folder_path' 2. ${NC}"
return 1
fi
# 2.
local protected_dirs=("/" "/home" "/usr" "/var" "/etc" "/root")
for protected in "${protected_dirs[@]}; do
if [ "$(realpath "$folder_path")" = "$protected" ]; then
echo -e "${RED}[2] 2. '$protected' 2. ${NC}"
return 1
fi
done
# 2.
echo -e "${YELLOW}--- 2. ---${NC}"
ls -la "$folder_path" 2>/dev/null || echo "(2.)"
# 2.
echo -e "${RED}[2] 2. ${NC}"
read -p "2. '$folder_path' 2. (2. yes 2.): " confirm
if [ "$confirm" != "yes" ]; then
echo -e "${YELLOW}2. ${NC}"
return 0
fi
echo -e "${YELLOW}[2.755]... ${NC}"
# -rf 2.755
rm -rf "$folder_path"
if [ $? -eq 0 ] && [ ! -d "$folder_path" ]; then
echo -e "${GREEN}[2] 2.755 '$folder_path' 2.755$parent_info${NC}"
log_operation "2.755" "$folder_path" "2.755"
else
echo -e "${RED}[2] 2.755$parent_info${NC}"
log_operation "2.755" "$folder_path" "2.755"
return 1
fi
}
# =====
# 3.
# 3.
# 1. 3.
# 2. 3.
# 3. 3. touch
# 4. 3.
# =====
create_file() {
echo -e "${BLUE}===== ${NC}"
echo -e "${BLUE} 3. ${NC}"
echo -e "${BLUE}===== ${NC}"

```



```
# █1██████████
echo -e "${YELLOW}--- ██████ ---${NC}"
read -p "██████████ (██████████): " target_dir
if [ -z "$target_dir" ]; then
target_dir=$(pwd)
fi
# ██████████
if [ ! -d "$target_dir" ]; then
echo -e "${RED}[██] ██ '$target_dir' █████${NC}"
return 1
fi
# █2██████████
read -p "██████████: " filename
# ██████████
if [ -z "$filename" ]; then
echo -e "${RED}[██] ██████████${NC}"
return 1
fi
# ██████████
file_path="$target_dir/$filename"
# ██████████
if [ -f "$file_path" ]; then
echo -e "${YELLOW}[██] ██ '$file_path' █████${NC}"
read -p "██████████(y/n): " overwrite
if [ "$overwrite" != "y" ] && [ "$overwrite" != "Y" ]; then
echo -e "${YELLOW}██████████${NC}"
return 0
fi
fi
# ████
echo -e "${YELLOW}[██████]...${NC}"
touch "$file_path"
if [ $? -eq 0 ]; then
# ██████████644██rw-r--r--██
chmod 644 "$file_path"
echo -e "${GREEN}[██] ██ '$file_path' █████${NC}"
# ██████████
read -p "██████████(y/n): " write_content
if [ "$write_content" = "y" ] || [ "$write_content" = "Y" ]; then
echo -e "${YELLOW}██████████ :wq █████${NC}"
echo "--- ██████ ---"
# ██████████
> "$file_path" # ████
while IFS= read -r line; do
if [ "$line" = ":wq" ]; then
break
fi
echo "$line" >> "$file_path"
done
echo "--- ██████ ---"
echo -e "${GREEN}██████████${NC}"
fi
# ██████████
echo -e "\n${YELLOW}--- ██████ ---${NC}"
ls -lh "$file_path"
echo "██████: $(stat -c%s "$file_path") ███"
echo "██████: $(stat -c%y "$file_path")"
log_operation "██████" "$file_path" "██"
else
echo -e "${RED}[██] ██████████${NC}"
log_operation "██████" "$file_path" "██"
return 1
fi
}
# =====
# █4██████/██████████
# ██████
# 1. ████████████████
# 2. ████████████████
# 3. ████████████████
# 4. ████████████████
# =====
modify_file() {
echo -e "${BLUE}===== ${NC}"
echo -e "${BLUE} ██████████${NC}"
echo -e "${BLUE}===== ${NC}"
}
```

```

# ██████████
read -p "██████████████████: " file_path
# ██████████
if [ ! -f "$file_path" ]; then
echo -e "${RED}[██] ██ '$file_path' █████${NC}"
return 1
fi
# ██████████
echo -e "\n${YELLOW}--- ██████ (10█) ---${NC}"
head -n 10 "$file_path"
echo "..."
echo -e "${YELLOW}██████: $(wc -l < "$file_path") █, $(stat -c%s "$file_path") █████${NC}"
echo ""
# ██████████
backup_path="$file_path.bak.$(date +%Y%m%d%H%M%S)"
cp "$file_path" "$backup_path"
echo -e "${GREEN}[██] ████████: $backup_path${NC}"
# ██████████
echo -e "${YELLOW}--- ██████ ---${NC}"
echo "1. ██████████"
echo "2. ██████████"
echo "3. ██████████"
echo "4. ██████████"
read -p "████ (1-4): " mode
case $mode in
1)
# ██████████ >> █████
echo -e "${YELLOW}██████████████████ :wq █████${NC}"
while IFS= read -r line; do
if [ "$line" = ":wq" ]; then
break
fi
echo "$line" >> "$file_path"
done
echo -e "${GREEN}[██] ██████████${NC}"
;;
2)
# ██████████ > █████
echo -e "${YELLOW}██████████████████ :wq █████${NC}"
> "$file_path" # ██████
while IFS= read -r line; do
if [ "$line" = ":wq" ]; then
break
fi
echo "$line" >> "$file_path"
done
echo -e "${GREEN}[██] ██████████${NC}"
;;
3)
# ██████████
read -p "██████████████: " line_num
echo -e "${YELLOW}██████████████████ :wq █████${NC}"
temp_file=$(mktemp)
line_count=1
inserted=0
while IFS= read -r line || [ -n "$line" ]; do
if [ "$line_count" = "$line_num" ] && [ "$inserted" -eq 0 ]; then
while IFS= read -r new_line; do
if [ "$new_line" = ":wq" ]; then
inserted=1
break
fi
echo "$new_line" >> "$temp_file"
done
fi
echo "$line" >> "$temp_file"
line_count=$((line_count + 1))
done < "$file_path"
mv "$temp_file" "$file_path"
echo -e "${GREEN}[██] ██████████ ${line_num} █████${NC}"
;;
4)
# ██████████sed███
read -p "██████████████████: " old_str
read -p "██████████████████: " new_str
# sed -i █████████s/old/new/g █████

```

```

sed -i "s/$old_str/$new_str/g" "$file_path"
echo -e "${GREEN}[██] ██ '$old_str' █████ '$new_str'${NC}"
;;
*)
echo -e "${RED}[██] █████${NC}"
;;
esac
log_operation "████" "$file_path" "██"
}
# =====
# █5████
# █████
# 1. █████
# 2. █████
# 3. █████
# =====
delete_file() {
echo -e "${BLUE}=====${NC}"
echo -e "${BLUE} █████${NC}"
echo -e "${BLUE}=====${NC}"
read -p "██████: " file_path
# ███████
if [ ! -f "$file_path" ]; then
echo -e "${RED}[██] ██ '$file_path' ███████${NC}"
return 1
fi
# █████
echo -e "${YELLOW}--- █████ ---${NC}"
ls -lh "$file_path"
# █████
system_dirs=("/etc/" "/usr/bin/" "/usr/sbin/")
for sys_dir in "${system_dirs[@]"; do
if [[ "$file_path" == $sys_dir* ]]; then
echo -e "${RED}[██] ███████${NC}"
break
fi
done
# ███
echo -e "${RED}[██] ███████${NC}"
read -p "██████ '$file_path' █(██ yes █): " confirm
if [ "$confirm" != "yes" ]; then
echo -e "${YELLOW}███████${NC}"
return 0
fi
# ███
rm -f "$file_path"
if [ $? -eq 0 ] && [ ! -f "$file_path" ]; then
echo -e "${GREEN}[██] ██ '$file_path' ███████${NC}"
log_operation "████" "$file_path" "██"
else
echo -e "${RED}[██] ███████${NC}"
log_operation "████" "$file_path" "██"
return 1
fi
}
# =====
# ███████
# =====
browse_directory() {
echo -e "${BLUE}=====${NC}"
echo -e "${BLUE} ███████${NC}"
echo -e "${BLUE}=====${NC}"
read -p "██████ (██████): " browse_path
if [ -z "$browse_path" ]; then
browse_path=$(pwd)
fi
if [ ! -d "$browse_path" ]; then
echo -e "${RED}[██] ███████${NC}"
return 1
fi
echo -e "\n${YELLOW}██: $browse_path${NC}"
echo ""
# ███████
echo "██ █ █ ███ ███"
echo "-----"
ls -lh "$browse_path" | tail -n +2 | while read -r line; do

```

[illegible]

MySQL

[illegible]

```

echo -e "${YELLOW}[██] █████...${NC}"
total_mem=$(free -m | awk '/Mem:/ {print $2}')
echo -e "${GREEN} █████: ${total_mem}MB${NC}"
if [ "$total_mem" -lt 512 ]; then
echo -e "${RED} [██] █████512MB██████MySQL██${NC}"
mysql_log "██: █████ ($total_mem)MB"
fi
mysql_log "████: ${total_mem}MB"
# 1.4 █████████MySQL████████500MB█
echo -e "${YELLOW}[██] █████...${NC}"
available_space=$(df -BG / | awk 'NR==2 {print $4}' | tr -d 'G')
echo -e "${GREEN} █████: ${available_space}GB${NC}"
if [ "$available_space" -lt 1 ]; then
echo -e "${RED} [██] ████████${NC}"
return 1
fi
mysql_log "██████: ${available_space}GB"
# 1.5 ████████
echo -e "${YELLOW}[██] █████...${NC}"
if ping -c 1 -W 3 mirrors.aliyun.com &>/dev/null; then
echo -e "${GREEN} ████████${NC}"
mysql_log "██████: ██"
else
echo -e "${RED} [██] ████████████████████${NC}"
mysql_log "██: ████████"
fi
echo -e "\n${GREEN}[██] ████████${NC}"
return 0
}
# =====
# ███2████████MySQL
# ██████
# 1. ████████████████████████
# 2. ███MySQL██YUM/APT██████████
# 3. █████████████████MySQL Server
# 4. ████████████████
# =====
install_mysql() {
echo -e "${CYAN}===== ${NC}"
echo -e "${CYAN} ███MySQL███${NC}"
echo -e "${CYAN}===== ${NC}"
mysql_log "[██2] █████MySQL..."
# ████████████████████████
case "$OS_NAME" in
*"Ubuntu"*|"Debian"*)
install_mysql_ubuntu
;;
*"CentOS"*|"Red Hat"*|"Rocky"*|"AlmaLinux"*)
install_mysql_centos
;;
*)
echo -e "${RED}[██] ██████████: $OS_NAME${NC}"
echo -e "${YELLOW}████████MySQL████████${NC}"
return 1
;;
esac
}
# Ubuntu/Debian████MySQL██
install_mysql_ubuntu() {
echo -e "${YELLOW}[Ubuntu/Debian] ████████...${NC}"
# ███apt██
sudo apt-get update -y
echo -e "${YELLOW}[████MySQL Server...]${NC}"
mysql_log "██apt██MySQL..."
# ███DEBIAN_FRONTEND=noninteractive██████████
# ███root██████████
sudo DEBIAN_FRONTEND=noninteractive apt-get install -y \
mysql-server mysql-client
if [ $? -eq 0 ]; then
echo -e "${GREEN}[██] MySQL████████${NC}"
mysql_log "MySQL██████ (Ubuntu/Debian)"
else
echo -e "${RED}[██] MySQL████████${NC}"
mysql_log "MySQL██████"
return 1
fi

```

```

}
# CentOS/RHELMySQL
install_mysql_centos() {
echo -e "${YELLOW}[CentOS/RHEL] MySQL YUM...${NC}"
# MySQL
# CentOS 7+MySQL
if [ ! -f /etc/yum.repos.d/mysql-community.repo ]; then
# MySQLMySQL 8.0
sudo yum install -y https://dev.mysql.com/get/mysql80-community-release-el7-11.noarch.rpm 2>/dev/null || \
{
echo -e "${YELLOW}MariaDB...${NC}"
# MySQLMariaDB
sudo yum install -y mariadb-server mariadb
if [ $? -eq 0 ]; then
echo -e "${GREEN}[ ] MariaDBMySQL${NC}"
mysql_log "MariaDB (CentOS)"
return 0
fi
}
fi
echo -e "${YELLOW}[ ] MySQL Server...${NC}"
mysql_log "yumMySQL..."
# MySQL Server
sudo yum install -y mysql-server mysql
if [ $? -eq 0 ]; then
echo -e "${GREEN}[ ] MySQL${NC}"
mysql_log "MySQL (CentOS)"
else
echo -e "${RED}[ ] MySQL${NC}"
mysql_log "MySQL"
return 1
fi
}
# =====
# 3MySQL
#
# 1. MySQLbinPATH
# 2. /etc/profile.d/mysql.sh
# 3. MySQLUTF-8
# 4. MySQL
# =====
configure_mysql_env() {
echo -e "${CYAN}=====${NC}"
echo -e "${CYAN} MySQL${NC}"
echo -e "${CYAN}=====${NC}"
mysql_log "[3] MySQL..."
# 3.1 MySQL
if command -v mysql &>/dev/null; then
MYSQL_BIN_DIR=$(dirname $(which mysql))
echo -e "${GREEN}MySQL: $MYSQL_BIN_DIR${NC}"
else
MYSQL_BIN_DIR="/usr/bin"
echo -e "${YELLOW}MySQL: $MYSQL_BIN_DIR${NC}"
fi
# 3.2
# /etc/profile.d/
ENV_FILE="/etc/profile.d/"
echo -e "${YELLOW}[ ] ...${NC}"
sudo tee "$ENV_FILE" > /dev/null << EOF
#!/bin/bash
# MySQL
# : $(date '+%Y-%m-%d %H:%M:%S')
# MySQLPATH
export PATH=$PATH:$MYSQL_BIN_DIR:$MYSQL_BIN_DIR/..sbin
# MySQL
export MYSQL_HOME=${MYSQL_BIN_DIR%/bin}
export DATADIR=$MYSQL_DATA_DIR
# MySQL
export MYSQL_PAGER=less
export MYSQL_EDITOR=vim
EOF
#
sudo chmod +x "$ENV_FILE"
# 3.3
source "$ENV_FILE"
export PATH=$PATH:$MYSQL_BIN_DIR

```

```

echo -e "${GREEN}[██] ████████${NC}"
echo -e " █████: $ENV_FILE"
mysql_log "██████████: $ENV_FILE"
# 3.4 ███MySQL██████
echo -e "${YELLOW}[██] ███MySQL████...${NC}"
sudo systemctl enable mysqld 2>/dev/null || sudo systemctl enable mariadb 2>/dev/null
echo -e "${GREEN}[██] MySQL██████████${NC}"
mysql_log "MySQL██████████"
}
# =====
# ███4████MySQL████root██
# █████
# 1. ███MySQL██
# 2. █████████████████mysql_secure_installation█
# 3. ███root████
# 4. ████████████████
# =====
initialize_mysql() {
echo -e "${CYAN}===== ${NC}"
echo -e "${CYAN} ███MySQL██ ${NC}"
echo -e "${CYAN}===== ${NC}"
mysql_log "[██4] ███MySQL..."
# 4.1 ███MySQL██
echo -e "${YELLOW}[██] MySQL██...${NC}"
sudo systemctl start mysqld 2>/dev/null || sudo systemctl start mariadb 2>/dev/null
sleep 3 # ████████
# █████
if sudo systemctl is-active --quiet mysqld 2>/dev/null || \
sudo systemctl is-active --quiet mariadb 2>/dev/null; then
echo -e "${GREEN}[██] MySQL██████${NC}"
mysql_log "MySQL██████████"
else
echo -e "${RED}[██] MySQL██████████${NC}"
mysql_log "MySQL██████████"
return 1
fi
# 4.2 █████████████████MySQL 8.0██████████████████
INITIAL_PASSWORD=""
if [ -f /var/log/mysqld.log ]; then
INITIAL_PASSWORD=$(sudo grep 'temporary password' /var/log/mysqld.log | awk '{print $NF}')
if [ -n "$INITIAL_PASSWORD" ]; then
echo -e "${YELLOW}[██] █████████████████${NC}"
fi
fi
# 4.3 ███root██
echo -e "${YELLOW}[██] ███root████...${NC}"
if [ -n "$INITIAL_PASSWORD" ]; then
# MySQL 8.0██████████████████
mysql -u root -p"$INITIAL_PASSWORD" --connect-expired-password \
-e "ALTER USER 'root'@'localhost' IDENTIFIED BY '$MYSQL_ROOT_PASSWORD';" 2>/dev/null
else
# MariaDB██████████
mysql -u root -e "ALTER USER 'root'@'localhost' IDENTIFIED BY '$MYSQL_ROOT_PASSWORD';" 2>/dev/null || \
mysqladmin -u root password "$MYSQL_ROOT_PASSWORD" 2>/dev/null
fi
if [ $? -eq 0 ]; then
echo -e "${GREEN}[██] root██████████${NC}"
echo -e " root██: $MYSQL_ROOT_PASSWORD"
mysql_log "root██████████"
else
echo -e "${YELLOW}[██] █████████████████...${NC}"
# █████████████████mysql.user█
sudo mysqld --skip-grant-tables --user=mysql &
sleep 2
mysql -u root -e "UPDATE mysql.user SET authentication_string=PASSWORD('$MYSQL_ROOT_PASSWORD') WHERE
User='root'; FLUSH PRIVILEGES;" 2>/dev/null || \
mysql -u root -e "SET PASSWORD FOR 'root'@'localhost' = PASSWORD('$MYSQL_ROOT_PASSWORD');" 2>/dev/null
sudo pkill -9 mysqld
sudo systemctl start mysqld 2>/dev/null || sudo systemctl start mariadb 2>/dev/null
mysql_log "██████████████"
fi
# 4.4 ████████████████████
echo -e "${YELLOW}[██] ████████...${NC}"
mysql -u root -p"$MYSQL_ROOT_PASSWORD" 2>/dev/null << EOF
-- ████████
DELETE FROM mysql.user WHERE User='';

```



```
-- root
DELETE FROM mysql.user WHERE User='root' AND Host NOT IN ('localhost', '127.0.0.1', '::1');
-- 
DROP DATABASE IF EXISTS test;
DELETE FROM mysql.db WHERE Db='test' OR Db='test\\_%';
-- 
FLUSH PRIVILEGES;
EOF
echo -e "${GREEN}[ ] MySQL${NC}"
mysql_log "MySQL"
}
# =====
# 5 MySQL
# 
# 1. root MySQL
# 2. MySQL
# 3. 
# =====
login_mysql_test() {
echo -e "${CYAN}=====${NC}"
echo -e "${CYAN} MySQL${NC}"
echo -e "${CYAN}=====${NC}"
mysql_log "[5] MySQL..."
echo -e "${YELLOW}[ ] MySQL...${NC}"
# MySQL
if mysql -u root -p"$MYSQL_ROOT_PASSWORD" -e "SELECT VERSION();" 2>/dev/null; then
echo -e "\n${GREEN}[ ] MySQL${NC}"
mysql_log "MySQL"
# MySQL
echo -e "\n${YELLOW}--- MySQL ---${NC}"
mysql -u root -p"$MYSQL_ROOT_PASSWORD" -e "SELECT VERSION() AS 'MySQL';" 2>/dev/null
# 
echo -e "\n${YELLOW}--- ---${NC}"
mysql -u root -p"$MYSQL_ROOT_PASSWORD" -e "SELECT USER() AS '';" 2>/dev/null
# 
echo -e "\n${YELLOW}--- ---${NC}"
sudo systemctl status mysqld 2>/dev/null | head -5 || sudo systemctl status mariadb 2>/dev/null | head -5
return 0
else
echo -e "${RED}[ ] MySQL${NC}"
mysql_log "MySQL"
return 1
fi
}
# =====
# 6 
# 
# 1. 
# 2. 
# 3. 
# 4. 
# =====
create_database_and_tables() {
echo -e "${CYAN}=====${NC}"
echo -e "${CYAN} ${NC}"
echo -e "${CYAN}=====${NC}"
mysql_log "[6] ..."
# 
DB_NAME="student_management"
echo -e "${YELLOW}[ ] : $DB_NAME${NC}"
# SQL
mysql -u root -p"$MYSQL_ROOT_PASSWORD" 2>/dev/null << EOSQL
-- =====
-- 
-- : $(date '+%Y-%m-%d %H:%M:%S')
-- =====
-- 
DROP DATABASE IF EXISTS $DB_NAME;
-- UTF-8
CREATE DATABASE $DB_NAME
DEFAULT CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
-- 
USE $DB_NAME;
-----
-- 1: students
```

```

-- ■■■■:
-- student_id: ■■■■■■■■■■
-- name: ■■
-- gender: ■■
-- age: ■■
-- class_name: ■■
-- phone: ■■■■
-- create_time: ■■■■■■
-----
CREATE TABLE students (
student_id INT AUTO_INCREMENT PRIMARY KEY COMMENT '■■',
name VARCHAR(50) NOT NULL COMMENT '■■■',
gender ENUM('■', '■') DEFAULT '■' COMMENT '■■■',
age INT CHECK (age BETWEEN 15 AND 30) COMMENT '■■■',
class_name VARCHAR(50) COMMENT '■■■',
phone VARCHAR(20) COMMENT '■■■■',
create_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP COMMENT '■■■■'
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='■■■■■';
-----
-- ■2: courses■■■■■■■
-- ■■■■:
-- course_id: ■■■■■■■■
-- course_name: ■■■■
-- credit: ■■
-- teacher: ■■■■
-----
CREATE TABLE courses (
course_id INT AUTO_INCREMENT PRIMARY KEY COMMENT '■■■■',
course_name VARCHAR(100) NOT NULL COMMENT '■■■■■',
credit DECIMAL(3,1) NOT NULL DEFAULT 1.0 COMMENT '■■■',
teacher VARCHAR(50) NOT NULL COMMENT '■■■■'
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='■■■■■';
-----
-- ■3: scores■■■■■
-- ■■■■:
-- score_id: ■■■■ID■■■
-- student_id: ■■■■■■■■students■■
-- course_id: ■■■■■■■■courses■■
-- score: ■■■0-100■
-- exam_time: ■■■■
-- ■■■■■■■■■■
-----
CREATE TABLE scores (
score_id INT AUTO_INCREMENT PRIMARY KEY COMMENT '■■■ID',
student_id INT NOT NULL COMMENT '■■■',
course_id INT NOT NULL COMMENT '■■■■',
score DECIMAL(5,2) CHECK (score BETWEEN 0 AND 100) COMMENT '■■■',
exam_time DATE COMMENT '■■■■',
-- ■■■■
CONSTRAINT fk_student FOREIGN KEY (student_id)
REFERENCES students(student_id) ON DELETE CASCADE,
CONSTRAINT fk_course FOREIGN KEY (course_id)
REFERENCES courses(course_id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='■■■';
EOSQL
if [ $? -eq 0 ]; then
echo -e "${GREEN}[■■] ■■■ '$DB_NAME' ■■■■■■■■${NC}"
mysql_log "■■■ $DB_NAME ■■■■"
else
echo -e "${RED}[■■] ■■■■■■■■${NC}"
mysql_log "■■■■■■■"
return 1
fi
}
# =====
# ■■7■■■■■■■
# ■■■■
# 1. ■■■■■■■■■■
# 2. ■■■■■■■■■■
# 3. ■■■■■■■■■■
# 4. ■■■■■■■■
# =====
insert_sample_data() {
echo -e "${CYAN}===== ${NC}"
echo -e "${CYAN} ■■■■■■${NC}"
echo -e "${CYAN}===== ${NC}"

```


[illegible]

■■■■Tomcat■■■■

```
#!/bin/bash
# =====
# ■■4■■Tomcat■■■■■■■■■■
# ■■■■■Tomcat■■■■■■■■■■■■■■■■■■■■
# ■■■■■■
# ■■■20231035109
# =====
# ■■■■
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
BLUE='\033[0;34m'
CYAN='\033[0;36m'
NC='\033[0m'
# Tomcat■■■■■
TOMCAT_VERSION="10.1.18"
TOMCAT_INSTALL_DIR="/opt"
TOMCAT_HOME="${TOMCAT_INSTALL_DIR}/tomcat-${TOMCAT_VERSION}"
TOMCAT_DOWNLOAD_URL="https://dlcdn.apache.org/tomcat/tomcat-10/v${TOMCAT_VERSION}/bin/apache-tomcat-${TOMCAT_V
ERSION}.tar.gz"
TOMCAT_ARCHIVE="apache-tomcat-${TOMCAT_VERSION}.tar.gz"
TOMCAT_LOG="/workspace/linux_course_design/tomcat_install.log"
# JDK■■■■Tomcat■■Java■■■■
JAVA_HOME=""
JDK_VERSION="11"
# ■■■■■■
init_tomcat_log() {
echo "===== " > "$TOMCAT_LOG"
echo "Tomcat■■■■■" >> "$TOMCAT_LOG"
echo "■■■■■: $(date '+%Y-%m-%d %H:%M:%S') " >> "$TOMCAT_LOG"
echo "===== " >> "$TOMCAT_LOG"
}
tomcat_log() {
local msg="$1"
local timestamp=$(date '+%Y-%m-%d %H:%M:%S')
echo "[$timestamp] $msg" | tee -a "$TOMCAT_LOG"
}
# =====
# ■■1■■■■Java■■■
# ■■■■■■
# 1. Tomcat■■■■■Java■■■■JDK/JRE■
# 2. ■■■■■■■■■Java
# 3. ■■■■■■■■■OpenJDK
# 4. ■■JAVA_HOME■■■■■
# =====
check_java_environment() {
echo -e "${CYAN}===== ${NC}"
echo -e "${CYAN} Java■■■■■${NC}"
echo -e "${CYAN}===== ${NC}"
tomcat_log "[■■1] ■■Java■■..."
# 1.1 ■■■■■Java
echo -e "${YELLOW}[■■] Java■■■■■...${NC}"
if command -v java >>/dev/null; then
JAVA_VERSION=$(java -version 2>&1 | head -n 1)
echo -e "${GREEN} ■■■: $JAVA_VERSION${NC}"
# ■■JAVA_HOME■■■
if [ -n "$JAVA_HOME" ]; then
echo -e "${GREEN} JAVA_HOME: $JAVA_HOME${NC}"
else
# ■■■■■■JAVA_HOME
JAVA_HOME=$(dirname $(dirname $(readlink -f $(which java))))
echo -e "${YELLOW} ■■■■■JAVA_HOME: $JAVA_HOME${NC}"
fi
tomcat_log "Java■■■: $JAVA_VERSION"
return 0
fi
echo -e "${RED} ■■■■Java■■■${NC}"
tomcat_log "■■■■Java■■■■■..."
# 1.2 ■■Java■■■■■■■■■■■■■■■■■■■■
read -p "■■■■■■■■OpenJDK■(y/n): " install_jdk
if [ "$install_jdk" = "y" ] || [ "$install_jdk" = "Y" ]; then
install_java
else
```

```

echo -e "${RED}[■■■] Tomcat■■■Java■■■■■■■■■■${NC}"
return 1
fi
}
# ■■■Java■■■
install_java() {
echo -e "${YELLOW}[■■■] OpenJDK...${NC}"
if [ -f /etc/os-release ]; then
. /etc/os-release
fi
case "$OS_NAME" in
*"Ubuntu"*|"Debian"*)
sudo apt-get update -y
sudo apt-get install -y openjdk-${JDK_VERSION}-jdk
;;
*"CentOS"*|"Red Hat"*|"Rocky"*|"AlmaLinux"*)
sudo yum install -y java-${JDK_VERSION}-openjdk-devel
;;
*)
echo -e "${RED}[■■■] ■■■■■■■■■${NC}"
return 1
;;
esac
if [ $? -eq 0 ]; then
# ■■■JAVA_HOME
export JAVA_HOME=$(dirname $(dirname $(readlink -f $(which java))))
echo -e "${GREEN}[■■■] Java■■■■■: $(java -version 2>&1 | head -n 1)${NC}"
tomcat_log "Java■■■■■, JAVA_HOME=$JAVA_HOME"
return 0
else
echo -e "${RED}[■■■] Java■■■■■${NC}"
return 1
fi
}
# =====
# ■■2■■■Tomcat
# ■■■■■
# 1. ■Apache■■■■■■■■■■Tomcat
# 2. ■■■■wget■■curl■■■■■
# 3. ■■■■■■■■■■■MD5/SHA■■■
# 4. ■■■■■■■■■
# =====
download_tomcat() {
echo -e "${CYAN}=====${NC}"
echo -e "${CYAN} ■■Tomcat${NC}"
echo -e "${CYAN}=====${NC}"
tomcat_log "[■■2] ■■Tomcat ${TOMCAT_VERSION}..."
# 2.1 ■■■■■■■■■
if [ -d "$TOMCAT_HOME" ]; then
echo -e "${YELLOW}Tomcat■■■■■■■: $TOMCAT_HOME${NC}"
read -p "■■■■■■■■■■(y/n): " redownload
if [ "$redownload" != "y" ] && [ "$redownload" != "Y" ]; then
echo -e "${YELLOW}■■■■■■■${NC}"
return 0
fi
# ■■■■■
sudo rm -rf "$TOMCAT_HOME"
fi
# 2.2 ■■■■■■■■■
TMP_DIR="/tmp/tomcat_install"
mkdir -p "$TMP_DIR"
cd "$TMP_DIR"
# ■■■■■■■■■
rm -f "$TOMCAT_ARCHIVE"
echo -e "${YELLOW}[■■■] Apache Tomcat ${TOMCAT_VERSION}...${NC}"
echo -e " ■■■■: $TOMCAT_DOWNLOAD_URL"
echo ""
# 2.3 ■■■wget■■curl■■■
DOWNLOAD_SUCCESS=0
if command -v wget >/dev/null; then
echo -e "${YELLOW}■■ wget ■■...${NC}"
wget --progress=bar:force "$TOMCAT_DOWNLOAD_URL" -O "$TOMCAT_ARCHIVE" && \
DOWNLOAD_SUCCESS=1
elif command -v curl >/dev/null; then
echo -e "${YELLOW}■■ curl ■■...${NC}"
curl -L --progress-bar "$TOMCAT_DOWNLOAD_URL" -o "$TOMCAT_ARCHIVE" && \

```

```

DOWNLOAD_SUCCESS=1
else
echo -e "${RED}[██] █████wget██curl██████${NC}"
echo -e "${YELLOW}████: sudo apt-get install wget █ sudo yum install wget${NC}"
return 1
fi
# 2.4 █████
if [ $DOWNLOAD_SUCCESS -eq 1 ] && [ -f "$TOMCAT_ARCHIVE" ]; then
FILE_SIZE=$(ls -lh "$TOMCAT_ARCHIVE" | awk '{print $5}')
echo -e "\n${GREEN}[██] Tomcat██████${NC}"
echo -e " █████: $FILE_SIZE"
echo -e " █████: $TMP_DIR/$TOMCAT_ARCHIVE"
tomcat_log "Tomcat████: $FILE_SIZE"
return 0
else
echo -e "\n${RED}[██] Tomcat██████${NC}"
echo -e "${YELLOW}██████: ${NC}"
echo " 1. █████"
echo " 2. █████"
echo " 3. █████"
tomcat_log "Tomcat████"
return 1
fi
}
# =====
# ███3██████Tomcat
# █████
# 1. █████tar.gz██████████
# 2. █████
# 3. █████Tomcat████████
# 4. █████
# =====
install_tomcat() {
echo -e "${CYAN}===== ${NC}"
echo -e "${CYAN} ███Tomcat${NC}"
echo -e "${CYAN}===== ${NC}"
tomcat_log "[██3] ███Tomcat..."
TMP_DIR="/tmp/tomcat_install"
ARCHIVE_PATH="$TMP_DIR/$TOMCAT_ARCHIVE"
# █████
if [ ! -f "$ARCHIVE_PATH" ]; then
echo -e "${RED}[██] ███Tomcat███: $ARCHIVE_PATH${NC}"
echo -e "${YELLOW}██████████████████2██${NC}"
return 1
fi
# 3.1 ███Tomcat████
echo -e "${YELLOW}[██] Tomcat███...${NC}"
sudo mkdir -p "$TOMCAT_INSTALL_DIR"
# ███tar██████-x███-z██gzip█-v██████-f████
sudo tar -xzf "$ARCHIVE_PATH" -C "$TOMCAT_INSTALL_DIR"
if [ $? -ne 0 ]; then
echo -e "${RED}[██] ████████████████${NC}"
tomcat_log "Tomcat████"
return 1
fi
# 3.2 █████
EXTRACTED_DIR="$TOMCAT_INSTALL_DIR/apache-tomcat-${TOMCAT_VERSION}"
if [ ! -d "$TOMCAT_HOME" ] && [ -d "$EXTRACTED_DIR" ]; then
sudo mv "$EXTRACTED_DIR" "$TOMCAT_HOME"
fi
# 3.3 ███
echo -e "${YELLOW}[██] █████...${NC}"
# ███Tomcat████████████████████tomcat███
sudo chown -R $USER:$USER "$TOMCAT_HOME"
# ███bin██████████
sudo chmod +x "$TOMCAT_HOME/bin/*.sh"
# 3.4 █████
if [ ! -L "/opt/tomcat" ]; then
sudo ln -s "$TOMCAT_HOME" /opt/tomcat
echo -e "${GREEN} █████: /opt/tomcat -> $TOMCAT_HOME${NC}"
fi
echo -e "\n${GREEN}[██] Tomcat██████${NC}"
echo -e " █████: $TOMCAT_HOME"
echo -e " ███: /opt/tomcat"
tomcat_log "Tomcat████: $TOMCAT_HOME"
# █████

```

```
echo -e "\n${YELLOW}--- Tomcat█████ --- ${NC}"  
ls -la "$TOMCAT_HOME/" | head -15  
}  
# =====  
# ███4████Tomcat█████  
# ██████  
# 1. ███CATALINA_HOME████████Tomcat█████  
# 2. ███PATH████Tomcat█bin███  
# 3. ████████████████████  
# 4. ███Tomcat███████████  
# =====  
configure_tomcat_env() {  
echo -e "${CYAN}===== ${NC}"  
echo -e "${CYAN} ███Tomcat███ ${NC}"  
echo -e "${CYAN}===== ${NC}"  
tomcat_log " [███4] ███Tomcat█████..."  
# 4.1 ███JAVA_HOME███████████  
if [ -z "$JAVA_HOME" ]; then  
if command -v java &>/dev/null; then  
export JAVA_HOME=$(dirname $(dirname $(readlink -f $(which java))))  
else  
echo -e "${RED}[███] ███Java███ ${NC}"  
return 1  
fi  
fi  
# 4.2 ███Tomcat███████████  
ENV_FILE="/etc/profile.d/tomcat.sh"  
echo -e "${YELLOW}[███] █████... ${NC}"  
sudo tee "$ENV_FILE" > /dev/null << EOF  
#!/bin/bash  
# Tomcat███████████  
# ████████: $(date +%Y-%m-%d %H:%M:%S')  
# Tomcat████████CATALINA_HOME████████Tomcat███████████  
export CATALINA_HOME=$TOMCAT_HOME  
export TOMCAT_HOME=$TOMCAT_HOME  
# ███Tomcat bin████████PATH████████startUp.sh█████  
export PATH=$PATH:$TOMCAT_HOME/bin  
# Java███████  
export JAVA_HOME=$JAVA_HOME  
EOF  
sudo chmod +x "$ENV_FILE"  
# ████████████  
source "$ENV_FILE"  
export CATALINA_HOME=$TOMCAT_HOME  
export PATH=$PATH:$TOMCAT_HOME/bin  
echo -e "${GREEN}[███] █████... ${NC}"  
echo -e " CATALINA_HOME=$TOMCAT_HOME"  
echo -e " █████: $ENV_FILE"  
tomcat_log " ████████: CATALINA_HOME=$TOMCAT_HOME"  
# 4.3 ███Tomcat███████████  
configure_tomcat_settings  
}  
# ███Tomcat███████████  
configure_tomcat_settings() {  
echo -e "\n${YELLOW}[███] Tomcat████████... ${NC}"  
SERVER_XML="$TOMCAT_HOME/conf/server.xml"  
if [ ! -f "$SERVER_XML" ]; then  
echo -e "${RED}[███] ███server.xml█████ ${NC}"  
return 1  
fi  
# ████████  
cp "$SERVER_XML" "${SERVER_XML}.bak"  
echo -e " ████████: ${SERVER_XML}.bak"  
# ████████████8080██████████80█  
read -p "██████Tomcat████████8080██(y/n): " change_port  
if [ "$change_port" = "y" ] || [ "$change_port" = "Y" ]; then  
read -p "██████████ (██8080/80/8888): " new_port  
if [ -n "$new_port" ] && [[ "$new_port" =~ ^[0-9]+$ ]]; then  
# ███sed████Connector█████  
sed -i "s/port=\"8080\"/port=\"$new_port\"/" "$SERVER_XML"  
echo -e "${GREEN} ████████: $new_port ${NC}"  
TOMCAT_PORT=$new_port  
fi  
else  
TOMCAT_PORT=8080  
fi  
}
```



```

# UTF-8
sed -i 's/URIEncoding=["^"]*/URIEncoding="UTF-8"/g' "$SERVER_XML"
echo -e " URI UTF-8"
echo -e "${GREEN}[ ] Tomcat${NC}"
tomcat_log "Tomcat, =${TOMCAT_PORT:-8080}"
}
# =====
# 5 Tomcat
#
# 1. Tomcat
# 2. startup.sh
# 3.
# 4.
# =====
start_tomcat() {
echo -e "${CYAN}=====${NC}"
echo -e "${CYAN} Tomcat${NC}"
echo -e "${CYAN}=====${NC}"
tomcat_log "[5] Tomcat..."
# 5.1 Tomcat
if [ ! -d "$TOMCAT_HOME" ]; then
echo -e "${RED}[ ] Tomcat${NC}"
return 1
fi
# 5.2
if pgrep -f "catalina" > /dev/null || pgrep -f "tomcat" > /dev/null; then
echo -e "${YELLOW}[ ] Tomcat${NC}"
read -p "Tomcat(y/n): " restart_choice
if [ "$restart_choice" = "y" ] || [ "$restart_choice" = "Y" ]; then
stop_tomcat_silent
else
echo -e "${YELLOW}${NC}"
return 0
fi
fi
# 5.3 Tomcat
echo -e "${YELLOW}[ ] Tomcat...${NC}"
cd "$TOMCAT_HOME"
# JAVA_HOME
export JAVA_HOME="${JAVA_HOME:-$(dirname $(dirname $(readlink -f $(which java))))}"
export CATALINA_HOME="$TOMCAT_HOME"
#
./bin/startup.sh
if [ $? -eq 0 ]; then
echo ""
echo -e "${GREEN}[ ] Tomcat${NC}"
# 5.4
echo -e "${YELLOW}[ ] ...${NC}"
sleep 5
# 5.5
check_tomcat_status
else
echo -e "${RED}[ ] Tomcat${NC}"
echo -e "${YELLOW}: $TOMCAT/logs/catalina.out${NC}"
tomcat_log "Tomcat"
return 1
fi
}
# Tomcat
stop_tomcat_silent() {
cd "$TOMCAT_HOME"
export JAVA_HOME="${JAVA_HOME:-$(dirname $(dirname $(readlink -f $(which java))))}"
./bin/shutdown.sh 2>/dev/null
sleep 2
#
pkill -9 -f catalina 2>/dev/null
}
# =====
# 6 Tomcat
#
# 1. Tomcat
# 2.
# 3. HTTP
# 4.
# =====
check_tomcat_status() {

```

```

echo -e "\n${YELLOW}--- Tomcat██████ ---${NC}\n"
PORT=${TOMCAT_PORT:-8080}
STATUS_OK=true
# 6.1 █████
echo -e "[██] █████..."
if pgrep -f ".*catalina.*$TOMCAT_HOME" > /dev/null || \
pgrep -f ".*org.apache.catalina.startup.Bootstrap" > /dev/null; then
PID=$(pgrep -f ".*org.apache.catalina.startup.Bootstrap" | head -1)
echo -e " ${GREEN}● █████ (PID: $PID)${NC}"
else
echo -e " ${RED}■ █████${NC}"
STATUS_OK=false
fi
# 6.2 ███████
echo -e "\n[██] █████..."
if netstat -tlnp 2>/dev/null | grep -q ":${PORT}" " || \
ss -tlnp 2>/dev/null | grep -q ":${PORT}"; then
echo -e " ${GREEN}● ██ $PORT █████${NC}"
else
echo -e " ${RED}■ ██ $PORT █████${NC}"
STATUS_OK=false
fi
# 6.3 HTTP████
echo -e "\n[██] HTTP████..."
if command -v curl &>/dev/null; then
HTTP_CODE=$(curl -s -o /dev/null -w "%{http_code}" http://localhost:$PORT/ --connect-timeout 5 2>/dev/null)
case $HTTP_CODE in
200|302)
echo -e " ${GREEN}● HTTP████ (████: $HTTP_CODE)${NC}"
;;
000)
echo -e " ${RED}■ █████${NC}"
STATUS_OK=false
;;
*)
echo -e " ${YELLOW}■ HTTP████: $HTTP_CODE${NC}"
;;
esac
fi
# 6.4 █████
echo ""
if [ "$STATUS_OK" = true ]; then
echo -e "${GREEN}=====${NC}"
echo -e "${GREEN} Tomcat██████${NC}"
echo -e "${GREEN} █████: http://localhost:$PORT${NC}"
echo -e "${GREEN}=====${NC}"
tomcat_log "Tomcat████, ██=http://localhost:$PORT"
return 0
else
echo -e "${RED}=====${NC}"
echo -e "${RED} Tomcat██████${NC}"
echo -e "${RED} █████: $TOMCAT_HOME/logs/catalina.out${NC}"
echo -e "${RED}=====${NC}"
tomcat_log "██: Tomcat████"
return 1
fi
}
# =====
# ███7███Tomcat██
# =====
stop_tomcat() {
echo -e "${CYAN}=====${NC}"
echo -e "${CYAN} ███Tomcat██${NC}"
echo -e "${CYAN}=====${NC}"
if [ ! -d "$TOMCAT_HOME" ]; then
echo -e "${RED}[██] Tomcat██████${NC}"
return 1
fi
echo -e "${YELLOW}[██] Tomcat██████...${NC}"
cd "$TOMCAT_HOME"
export JAVA_HOME="$(dirname $(dirname $(readlink -f $(which java))))"
./bin/shutdown.sh
sleep 3
if pgrep -f "catalina" > /dev/null; then
echo -e "${YELLOW}[██] ████████████████...${NC}"
pkill -9 -f catalina

```

```

sleep 1
fi
if ! pgrep -f "catalina" > /dev/null; then
echo -e "${GREEN}[██] Tomcat███${NC}"
tomcat_log "Tomcat███"
else
echo -e "${RED}[██] Tomcat██████${NC}"
return 1
fi
}
# =====
# █████
# =====
main_menu() {
init_tomcat_log
while true; do
echo ""
echo -e "${GREEN}██████████████████████████████████████████████████████████████${NC}"
echo -e "${GREEN}██ Tomcat ██████████ v1.0 █${NC}"
echo -e "${GREEN}██████████████████████████████████████████████████████████████${NC}"
echo -e "${GREEN}██ █${NC}"
echo -e "${GREEN}██ 1. ███Java███ █${NC}"
echo -e "${GREEN}██ 2. ███Tomcat █${NC}"
echo -e "${GREEN}██ 3. ███Tomcat █${NC}"
echo -e "${GREEN}██ 4. ████████ █${NC}"
echo -e "${GREEN}██ 5. ███Tomcat███ █${NC}"
echo -e "${GREEN}██ 6. ████████ █${NC}"
echo -e "${GREEN}██ 7. ███Tomcat███ █${NC}"
echo -e "${GREEN}██ 8. ████████ █${NC}"
echo -e "${GREEN}██ 0. ██████ █${NC}"
echo -e "${GREEN}██ █${NC}"
echo -e "${GREEN}██████████████████████████████████████████████████████████████${NC}"
echo ""
read -p "██████ (0-8): " choice
case $choice in
1)
check_java_environment
;;
2)
download_tomcat
;;
3)
install_tomcat
;;
4)
configure_tomcat_env
;;
5)
start_tomcat
;;
6)
check_tomcat_status
;;
7)
stop_tomcat
;;
8)
echo -e "${YELLOW}[████] ██████████...${NC}"
check_java_environment && \
download_tomcat && \
install_tomcat && \
configure_tomcat_env && \
start_tomcat
echo -e "${GREEN}[██] ██████████${NC}"
;;
0)
echo -e "${YELLOW}██████Tomcat██████████${NC}"
exit 0
;;
*)
echo -e "${RED}[██] ████████████████████${NC}"
;;
esac
echo ""
read -p "██████████..."
done

```

```
}  
# ■■■■  
main_menu
```

Web

[illegible]

```

echo -e " ${GREEN}● ■■■■: $TOMCAT_HOME${NC}"
if pgrep -f "catalina" > /dev/null || \
pgrep -f "org.apache.catalina.startup.Bootstrap" > /dev/null; then
echo -e " ${GREEN}● ■■■■: ■■■${NC}"
deploy_log "Tomcat: OK - ■■■"
else
echo -e " ${YELLOW}■ ■■■■: ■■■${NC}"
deploy_log "Tomcat: WARN - ■■■"
fi
else
echo -e " ${RED}■ ■■■ (■■: $TOMCAT_HOME)${NC}"
all_ok=false
deploy_log "Tomcat: FAIL - ■■■"
fi
# 1.3 ■■MySQL
echo -e "\n${YELLOW}[3/5] MySQL■■■...${NC}"
if command -v mysql &>/dev/null; then
if mysql -u root -p"$MYSQL_ROOT_PASSWORD" -e "SELECT 1;" &>/dev/null; then
echo -e " ${GREEN}● ■■■■■■■■${NC}"
deploy_log "MySQL: OK - ■■■"
else
echo -e " ${YELLOW}■ ■■■■■■■■■■■■■■■■■■■■${NC}"
deploy_log "MySQL: WARN - ■■■"
fi
else
echo -e " ${RED}■ ■■■■■■■■PATH■${NC}"
all_ok=false
deploy_log "MySQL: FAIL - ■■■"
fi
# 1.4 ■■■■■■■■■■■■■■■■■■■■100MB■
echo -e "\n${YELLOW}[4/5] ■■■■...${NC}"
available_mb=$(df -BM /opt | awk 'NR==2 {print $4}' | tr -d 'M')
if [ "$available_mb" -gt 100 ]; then
echo -e " ${GREEN}● /opt ■■■■: ${available_mb}MB${NC}"
deploy_log "■■■■: OK - ${available_mb}MB"
else
echo -e " ${RED}■ ■■■■: ■■ ${available_mb}MB${NC}"
all_ok=false
deploy_log "■■■■: FAIL - ${available_mb}MB"
fi
# 1.5 ■■■■■■■■■■■■■■■■■■■■
echo -e "\n${YELLOW}[5/5] ■■■■...${NC}"
if ping -c 1 -W 2 localhost &>/dev/null; then
echo -e " ${GREEN}● ■■■■■■${NC}"
deploy_log "■■: OK"
else
echo -e " ${RED}■ ■■■■${NC}"
deploy_log "■■: FAIL"
fi
# ■■■■
echo ""
echo -e "${PURPLE}===== ${NC}"
if [ "$all_ok" = true ]; then
echo -e "${GREEN} ■■■■■■ ✓${NC}"
deploy_log "■■■■■: ■■"
return 0
else
echo -e "${RED} ■■■■■■ X${NC}"
echo -e "${YELLOW} ■■■■■■■■■■■■■■■■■■■■${NC}"
deploy_log "■■■■■: ■■■"
return 1
fi
}
# =====
# ■■■■■■■■■■■■■■■■■■■■
# ■■■■■■
# 1. ■■■■■■■■■■■■■■■■■■■■
# 2. ■■■■■■Tomcatwebapps■■
# 3. ■■Java Web■■■■■WAR■■■
# 4. ■■■■■■■■■■
# =====
copy_project_code() {
echo -e "${CYAN}===== ${NC}"
echo -e "${CYAN} ■■■■■■${NC}"
echo -e "${CYAN}===== ${NC}"
deploy_log "[■■2] ■■■■■■..."

```

[illegible]

```
<link rel="stylesheet" href="css/style.css">
</head>
<body>
<div class="container">
<header>
<h1>■■■■■■■</h1>
<p class="subtitle">Linux■■■■■■■■■■■■■■■■■■■■</p>
</header>
<nav class="navbar">
<a href="index.html" class="active">■■■</a>
<a href="student_list.html">■■■■■</a>
<a href="about.html">■■■■■</a>
</nav>
<main>
<section class="welcome-section">
<h2>■■■■■■■■■■■■■■■■■■■■</h2>
<p>■■■■■■■Linux Shell■■■■■■■Web■■■■■■■■■■■■■■■■■■■■</p>
<div class="info-cards">
<div class="card">
<h3>■■■</h3>
<ul>
<li>Linux■■■■■</li>
<li>Shell■■■■■</li>
<li>Apache Tomcat</li>
<li>MySQL■■■</li>
</ul>
</div>
<div class="card">
<h3>■■■■■</h3>
<ul>
<li>■■■■■</li>
<li>■■■■■</li>
<li>■■■■■</li>
<li>■■■■■</li>
</ul>
</div>
</div>
<section class="status-section">
<h2>■■■■■</h2>
<table class="status-table">
<tr><td>■■■■</td><td class="success">■■■■■</td></tr>
<tr><td>■■■■</td><td class="success">■■■■■</td></tr>
<tr><td>■■■■■</td><td id="deploy-time"></td></tr>
<tr><td>■■■■■</td><td>v1.0.0</td></tr>
</table>
</section>
</main>
<footer>
<p>&copy; 2024 Linux■■■■■■■■■■ | ■■■■■■</p>
</footer>
</div>
<script src="js/main.js"></script>
</body>
</html>
HTMLEOF
# =====
# ■■■■■■■■ student_list.html
# =====
cat > "$PROJECT_DEPLOY_DIR/student_list.html" << 'HTMLEOF'
<!DOCTYPE html>
<html lang="zh-CN">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>■■■ - ■■■■■■</title>
<link rel="stylesheet" href="css/style.css">
</head>
<body>
<div class="container">
<header>
<h1>■■■■■■■</h1>
</header>
<nav class="navbar">
<a href="index.html">■■■</a>
<a href="student_list.html" class="active">■■■■■</a>
```



```


```

[illegible]

```

color: #667eea;
margin-bottom: 20px;
padding-bottom: 10px;
border-bottom: 2px solid #667eea;
}
.info-cards {
display: grid;
grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
gap: 20px;
margin-top: 20px;
}
.card {
background: #f9f9f9;
padding: 20px;
border-radius: 10px;
box-shadow: 0 2px 10px rgba(0,0,0,0.1);
}
.card h3 {
color: #764ba2;
margin-bottom: 15px;
}
.card ul {
list-style: none;
padding-left: 0;
}
.card ul li {
padding: 8px 0;
border-bottom: 1px solid #eee;
}
.card ul li:before {
content: "✓ ";
color: #667eea;
font-weight: bold;
}
.status-table {
width: 100%;
max-width: 500px;
border-collapse: collapse;
margin-top: 20px;
}
.status-table td {
padding: 12px 15px;
border: 1px solid #ddd;
}
.status-table td:first-child {
font-weight: bold;
background: #f5f5f5;
width: 40%;
}
.success {
color: #28a745;
font-weight: bold;
}
.data-table {
width: 100%;
border-collapse: collapse;
margin-top: 20px;
background: white;
box-shadow: 0 2px 10px rgba(0,0,0,0.1);
}
.data-table th,
.data-table td {
padding: 12px 15px;
text-align: left;
border-bottom: 1px solid #ddd;
}
.data-table th {
background: #667eea;
color: white;
font-weight: bold;
}
.data-table tr:hover {
background: #f5f5f5;
}
.toolbar {
margin-bottom: 20px;
}

```

```

display: flex;
gap: 10px;
}
.toolbar input {
flex: 1;
padding: 10px 15px;
border: 1px solid #ddd;
border-radius: 5px;
font-size: 14px;
}
.toolbar button {
padding: 10px 25px;
background: #667eea;
color: white;
border: none;
border-radius: 5px;
cursor: pointer;
transition: background 0.3s;
}
.toolbar button:hover {
background: #764ba2;
}
.pagination {
margin-top: 20px;
display: flex;
justify-content: center;
align-items: center;
gap: 15px;
}
.pagination button {
padding: 8px 20px;
background: #667eea;
color: white;
border: none;
border-radius: 5px;
cursor: pointer;
}
.about-content {
line-height: 1.8;
}
.about-content h3 {
color: #667eea;
margin: 25px 0 15px;
font-size: 1.3em;
}
.about-content ul,
.about-content ol {
margin-left: 25px;
margin-top: 10px;
}
.about-content li {
margin-bottom: 8px;
}
footer {
background: #333;
color: white;
text-align: center;
padding: 20px;
margin-top: 40px;
}
CSSEOF
# =====
# ■■■JavaScript■■■ js/main.js
# =====
cat > "$PROJECT_DEPLOY_DIR/js/main.js" << 'JSEOF'
// ■■■■■ JavaScript
// ■■■■■■■■■■
document.addEventListener('DOMContentLoaded', function() {
var timeElement = document.getElementById('deploy-time');
if (timeElement) {
timeElement.textContent = new Date().toLocaleString('zh-CN');
}
// ■■■■■■■■■■
loadStudentData();
});
// ■■■■■■■■■■■■■■■■■■■■■API■■■■■

```

```

var studentsData = [
{ id: 1, name: '■■■', gender: '■', age: 20, className: '■■■■2301' },
{ id: 2, name: '■■■', gender: '■', age: 19, className: '■■■■2301' },
{ id: 3, name: '■■■', gender: '■', age: 21, className: '■■■■2302' },
{ id: 4, name: '■■■', gender: '■', age: 20, className: '■■■■2302' },
{ id: 5, name: '■■■', gender: '■', age: 19, className: '■■■■2301' }
];
var currentPage = 1;
var pageSize = 5;
// ■■■■■■■■■■
function loadStudentData() {
var tbody = document.getElementById('student-tbody');
if (!tbody) return;
tbody.innerHTML = '';
studentsData.forEach(function(student) {
var row = document.createElement('tr');
row.innerHTML =
'<td>' + student.id + '</td>' +
'<td>' + student.name + '</td>' +
'<td>' + student.gender + '</td>' +
'<td>' + student.age + '</td>' +
'<td>' + student.className + '</td>' +
'<td><button onclick="viewDetail(' + student.id + ')">■■■</button></td>';
tbody.appendChild(row);
});
}
// ■■■■
function searchStudents() {
var keyword = document.getElementById('search-input').value.toLowerCase();
alert('■■■■■' + (keyword || '■■■'));
}
// ■■■■
function prevPage() {
if (currentPage > 1) {
currentPage--;
updatePageInfo();
}
}
function nextPage() {
currentPage++;
updatePageInfo();
}
function updatePageInfo() {
var pageInfo = document.getElementById('page-info');
if (pageInfo) {
pageInfo.textContent = '■ ' + currentPage + ' ■';
}
}
// ■■■■
function viewDetail(id) {
alert('■■■■■ID: ' + id + ' ■■■■■');
}
JSEOF
# =====
# ■■WEB-INF■■■■ web.xml
# =====
cat > "$PROJECT_DEPLOY_DIR/WEB-INF/web.xml" << 'XMLEOF'
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
version="4.0">
<display-name>■■■■■■■</display-name>
<description>
Linux■■■■■■■ - ■■■■■■■■■■
</description>
<welcome-file-list>
<welcome-file>index.html</welcome-file>
<welcome-file>index.htm</welcome-file>
</welcome-file-list>
</web-app>
XMLEOF
# ■■■■
chmod -R 755 "$PROJECT_DEPLOY_DIR"
# ■■■■■■■■

```

```

file_count=$(find "$PROJECT_DEPLOY_DIR" -type f | wc -l)
echo -e "${GREEN}[██] ██████████${NC}"
echo -e " ██████: $PROJECT_NAME"
echo -e " ██████: $PROJECT_DEPLOY_DIR"
echo -e " ██████: $file_count"
echo -e "\n${YELLOW}--- ██████ ---${NC}"
tree "$PROJECT_DEPLOY_DIR" 2>/dev/null || find "$PROJECT_DEPLOY_DIR" -type f | head -20
deploy_log "██████████: $PROJECT_DEPLOY_DIR ($file_count ██████)"
}
# =====
# ███3████MySQL██
# ██████
# 1. ███MySQL████
# 2. ██████████
# 3. ████████████████
# 4. ████████
# =====
start_mysql_service() {
echo -e "${CYAN}===== ${NC}"
echo -e "${CYAN} ███MySQL████${NC}"
echo -e "${CYAN}===== ${NC}"
deploy_log "[████3] ███MySQL██..."
# 3.1 ███MySQL████
echo -e "${YELLOW}[██] MySQL████...${NC}"
if sudo systemctl is-active --quiet mysqld 2>/dev/null || \
sudo systemctl is-active --quiet mariadb 2>/dev/null; then
echo -e "${GREEN} MySQL██████${NC}"
else
echo -e "${YELLOW}[██] MySQL██...${NC}"
sudo systemctl start mysqld 2>/dev/null || sudo systemctl start mariadb 2>/dev/null
sleep 3
if sudo systemctl is-active --quiet mysqld 2>/dev/null || \
sudo systemctl is-active --quiet mariadb 2>/dev/null; then
echo -e "${GREEN} MySQL██████${NC}"
deploy_log "MySQL██████"
else
echo -e "${RED}[██] MySQL██████${NC}"
deploy_log "MySQL██████"
return 1
fi
fi
# 3.2 ██████
echo -e "\n${YELLOW}[██] ██████...${NC}"
if mysql -u root -p"$MYSQL_ROOT_PASSWORD" -e "SELECT 'Connection OK';" 2>/dev/null; then
echo -e "${GREEN} ██████${NC}"
deploy_log "██████████"
else
echo -e "${RED}[██] ████████████████████${NC}"
deploy_log "██████████"
fi
# 3.3 ████████████████████
echo -e "\n${YELLOW}[██] ██████...${NC}"
DB_EXISTS=$(mysql -u root -p"$MYSQL_ROOT_PASSWORD" -e "SHOW DATABASES LIKE '$DB_NAME';" 2>/dev/null | wc -l)
if [ "$DB_EXISTS" -gt 0 ]; then
echo -e "${GREEN} ███ '$DB_NAME' ███${NC}"
else
echo -e "${YELLOW}[██] ██████...${NC}"
# ████████████████████SQL
echo -e "${GREEN} ██████${NC}"
deploy_log "██████████"
fi
return 0
}
# =====
# ███4████Tomcat██
# ██████
# 1. ███Tomcat████
# 2. ███startup.sh██Tomcat
# 3. ████████████████
# 4. ███HTTP████
# =====
start_tomcat_service() {
echo -e "${CYAN}===== ${NC}"
echo -e "${CYAN} ███Tomcat████${NC}"
echo -e "${CYAN}===== ${NC}"
deploy_log "[████4] ███Tomcat██..."

```

```

# 4.1 ■■Tomcat■■■■■
if [ ! -d "$TOMCAT_HOME" ]; then
echo -e "${RED}[■■] Tomcat■■■■: $TOMCAT_HOME${NC}"
echo -e "${YELLOW}■■■■■4■Tomcat■■■■${NC}"
return 1
fi
# 4.2 ■■■■Tomcat
echo -e "${YELLOW}[■■] Tomcat■■■...${NC}"
if pgrep -f "org.apache.catalina.startup.Bootstrap" > /dev/null; then
echo -e "${GREEN} Tomcat■■■■■${NC}"
# ■■■■■■■■■■
read -p "■■■■■■■■■■■■■■■■■■■■Tomcat■(y/n): " restart_tc
if [ "$restart_tc" = "y" ] || [ "$restart_tc" = "Y" ]; then
restart_tomcat
fi
else
echo -e "${YELLOW}[■■] Tomcat■■■...${NC}"
cd "$TOMCAT_HOME"
export JAVA_HOME=$(dirname $(dirname $(readlink -f $(which java))))
./bin/startup.sh
if [ $? -eq 0 ]; then
echo -e "${GREEN} Tomcat■■■■■${NC}"
else
echo -e "${RED}[■■] Tomcat■■■■■${NC}"
return 1
fi
fi
# 4.3 ■■Tomcat■■■■■
echo -e "\n${YELLOW}[■■] ■■■■■■■10■■■...${NC}"
sleep 10
# 4.4 ■■■■■■
check_tomcat_running
}
# ■■Tomcat
restart_tomcat() {
echo -e "${YELLOW}[■■] ■■■■Tomcat...${NC}"
cd "$TOMCAT_HOME"
export JAVA_HOME=$(dirname $(dirname $(readlink -f $(which java))))
./bin/shutdown.sh 2>/dev/null
sleep 3
kill -9 -f catalina 2>/dev/null
sleep 2
./bin/startup.sh
sleep 10
echo -e "${GREEN} Tomcat■■■■■${NC}"
deploy_log "Tomcat■■■■"
}
# ■■Tomcat■■■■■
check_tomcat_running() {
PORT=${TOMCAT_PORT:-8080}
PROJECT_URL="http://localhost:$PORT/$PROJECT_NAME/"
echo -e "\n${YELLOW}--- ■■■■■■ ---${NC}\n"
# ■■■■
if pgrep -f "org.apache.catalina.startup.Bootstrap" > /dev/null; then
echo -e " ${GREEN}● ■■: ■■■${NC}"
else
echo -e " ${RED}■ ■■: ■■■${NC}"
return 1
fi
# ■■■■
if ss -tlnp 2>/dev/null | grep -q ":$PORT "; then
echo -e " ${GREEN}● ■■ $PORT: ■■■${NC}"
else
echo -e " ${RED}■ ■■ $PORT: ■■■${NC}"
fi
# HTTP■■■
if command -v curl &>/dev/null; then
HTTP_CODE=$(curl -s -o /dev/null -w "%{http_code}" http://localhost:$PORT/ --connect-timeout 5 2>/dev/null)
if [ "$HTTP_CODE" = "200" ] || [ "$HTTP_CODE" = "302" ]; then
echo -e " ${GREEN}● HTTP: ■■■■ ($HTTP_CODE)${NC}"
else
echo -e " ${RED}■ HTTP: ■■■■ ($HTTP_CODE)${NC}"
fi
fi
echo ""
echo -e "${GREEN}===== ${NC}"

```


[illegible]

```
read -p "■■■■■ (0-6): " choice
case $choice in
1)
check_deploy_environment
;;
2)
copy_project_code
;;
3)
start_mysql_service
;;
4)
start_tomcat_service
;;
5)
test_project_access
;;
6)
full_deploy
;;
0)
echo -e "${YELLOW}■■■■■■■■■■■■■■■■■■■■${NC}"
exit 0
;;
*)
echo -e "${RED}[■■] ■■■■■■■■■■■■■■■■■■■■${NC}"
;;
esac
echo ""
read -p "■■■■■■■■..."
done
}
# ■■■■
main_menu
```